



uOttawa

Lab 1 - Introduction to VHDL and ASM Design

CEG3155-Digital Systems II

Fall 2026

School of Electrical Engineering and Computer Science
University of Ottawa

Course Coordinator:
Prof. Rami Abielmona

Teaching Assistants:

[TA name]

[TA name]

[TA name]

Group: [#]

Student Name and number: Marcel Traore 300379484

Student Name and number: Kai Rasco #300304789

Experiment Date: [Date]

Submission Date: [Date]

Contents

1	Objective	2
2	Problem Description	2
3	Design	2
3.1	Pseudocode / Flowchart	2
3.2	ASM Chart	3
3.3	Datapath	4
3.4	Detailed ASM Chart	5
3.5	Control Logic (One-FF-Per-State)	6
4	Implementation	7
4.1	Component Descriptions	7
4.2	Planned Pin Assignment	7
5	Simulation Results	8
5.1	Shift Register	8
5.2	Control Logic	9
5.3	Full Display Controller	9
6	Hardware Test	9
7	Design Obstacles	9
8	Conclusion	9
A	Appendix: VHDL Source Code	10
A.1	dFF_2.vhd	10
A.2	lab-1-control.vhdl	11
A.3	dFlipFlop.vhdl	12
A.4	mux4_1bit.vhdl	13
A.5	shiftReg-nbit.vhdl	13

1 Objective

The objective of this lab is to design a light display controller using the ASM method and implement it in structural VHDL on the Nexys A7-100T (Artix-7) board.

2 Problem Description

The goal is a controller for a row of eight LEDs driven by two switches. When only LEFT is on, a single light moves from right to left. When only RIGHT is on, a single light moves from left to right. When both are on, both lights move at the same time and cross each other. When both are off, the display is blank, but the position of each light is kept, so turning a switch back on resumes the motion from where it stopped. GReset brings the whole circuit back to its initial state. The lights move one position per second so the motion is visible.

Port	Name	Description
Input	GClock	Global clock
Input	GReset	Global reset
Input	Right	Right switch: 0 = off, 1 = on
Input	Left	Left switch: 0 = off, 1 = on
Output	DisplayOut[7..0]	Eight display LEDs

Table 1: Input/output specification

3 Design

3.1 Pseudocode / Flowchart

The design starts from the pseudocode given in the lab manual:

- $DISPLAY \leftarrow 00000000$; $LMASK \leftarrow 00000001$; $RMASK \leftarrow 10000000$;
- if (LEFT and RIGHT)
 - $DISPLAY \leftarrow LMASK \vee RMASK$;
 - $LMASK \leftarrow LMASK \ll 1$;
 - $RMASK \leftarrow RMASK \gg 1$;
- else if (LEFT)
 - $DISPLAY \leftarrow LMASK$;
 - $LMASK \leftarrow LMASK \ll 1$;
- else if (RIGHT)
 - $DISPLAY \leftarrow RMASK$;
 - $RMASK \leftarrow RMASK \gg 1$;
- else
 - $DISPLAY \leftarrow 00000000$;
- Go back to first if statement;

The ASM chart in Figure 1 also serves as the flowchart of the solution.

3.2 ASM Chart

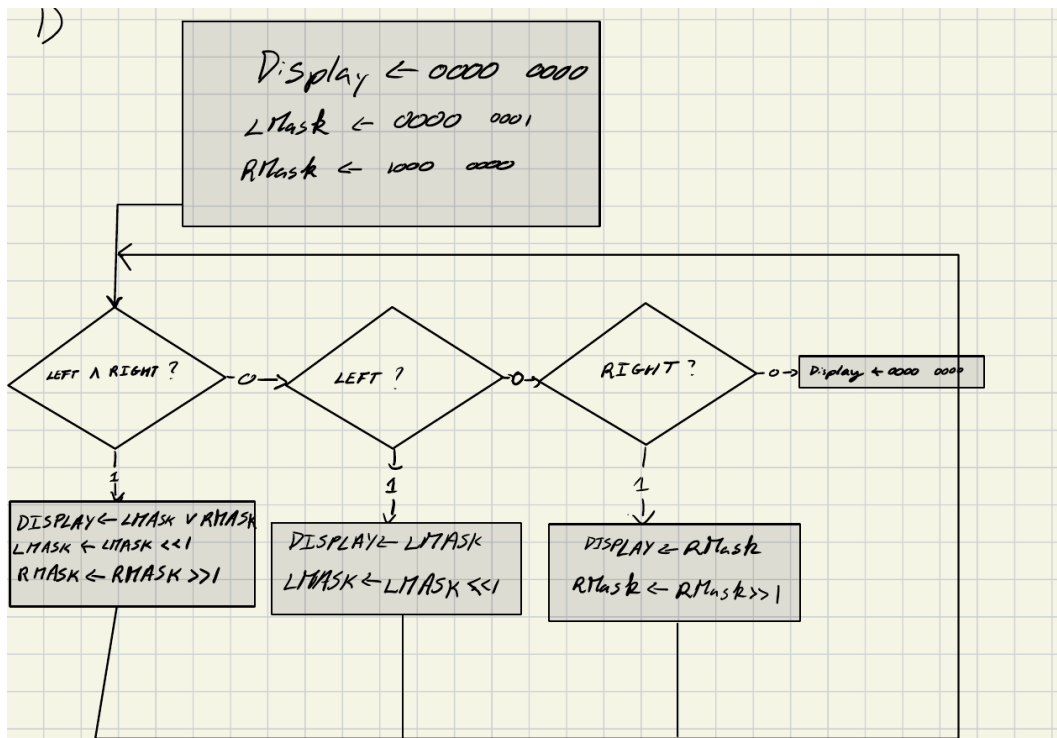


Figure 1: ASM chart

The chart has five states. S0 is the initial state: DISPLAY is cleared, LMASK is set to 00000001 and RMASK to 10000000. From every state the chart returns to the same chain of decisions. If both switches are on (S1), DISPLAY takes LMASK OR RMASK and both masks shift. If only LEFT is on (S2), DISPLAY takes LMASK and LMASK shifts left. If only RIGHT is on (S3), DISPLAY takes RMASK and RMASK shifts right. If neither is on (S4), DISPLAY is cleared. The masks are not reset in S4, so the lights resume from where they stopped.

3.3 Datapath

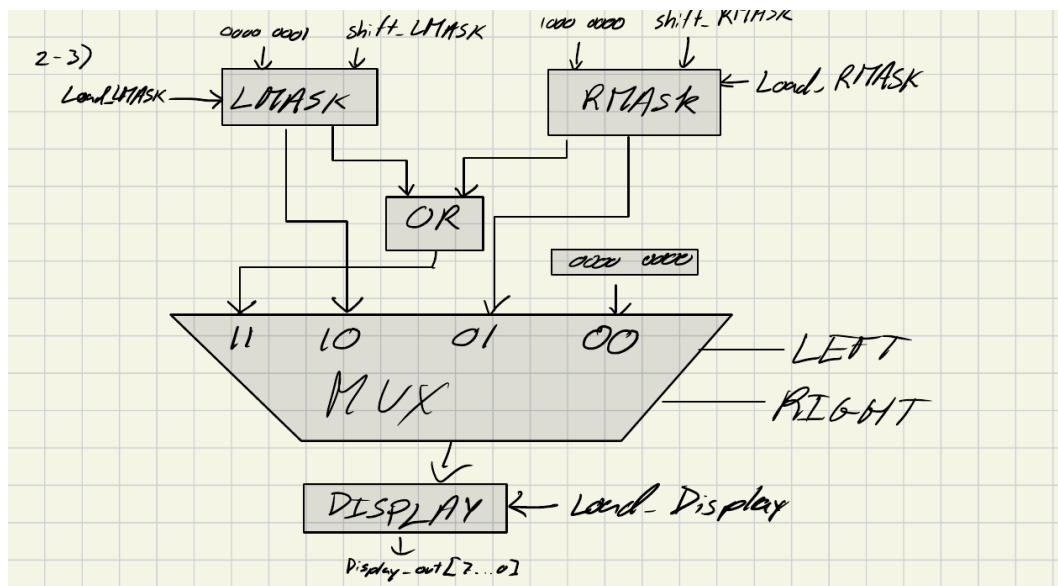


Figure 2: Datapath

Following the ASM rules, each unique name in the RTN statements gets its own register: LMASK, RMASK and DISPLAY. LMASK and RMASK are shift registers with a parallel load of their initial value (00000001 and 10000000) and control inputs Load_LMASK/shift_LMASK and Load_RMASK/shift_RMASK. An 8-bit OR gate forms LMASK OR RMASK. Since DISPLAY has four possible sources, a 4-to-1 mux selects its input, with LEFT and RIGHT used directly as the select lines: 11 selects LMASK OR RMASK, 10 selects LMASK, 01 selects RMASK and 00 selects 00000000. DISPLAY is loaded when Load_Display is high and drives DisplayOut[7..0].

Because the mux select comes directly from the switches, loading DISPLAY in S0 does not guarantee 00000000 unless both switches are off. In a complete implementation, DISPLAY should instead be cleared by the global reset (or the mux select forced to 00 in S0).

3.4 Detailed ASM Chart

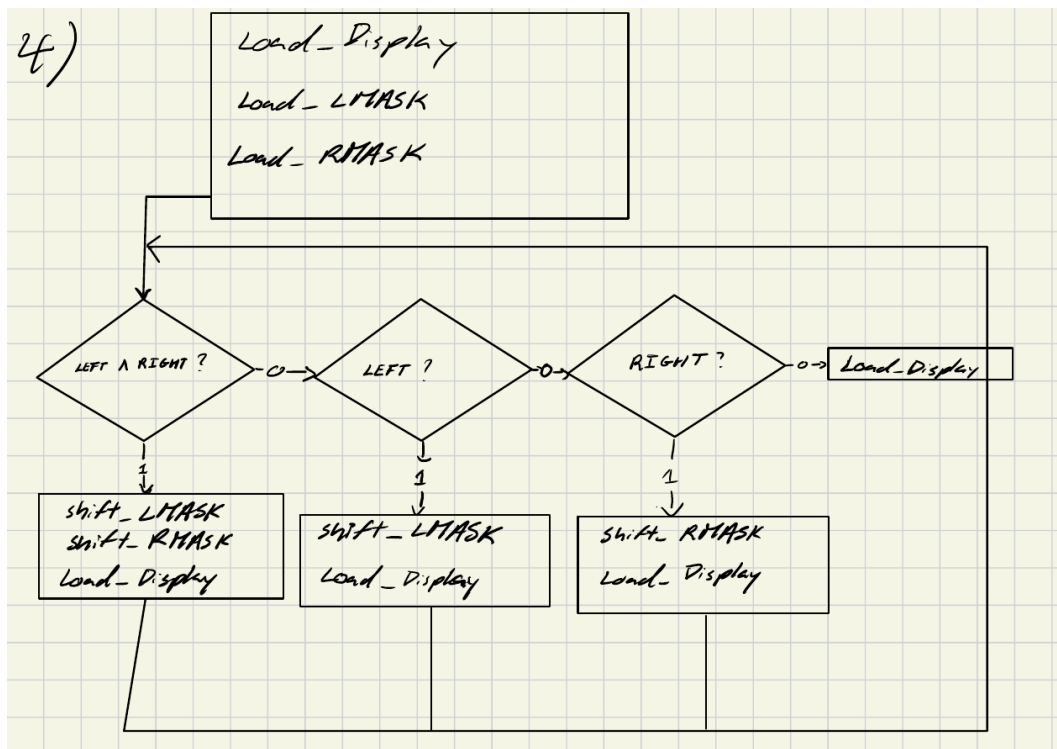


Figure 3: Detailed ASM chart

Each RTN statement of Figure 1 is replaced by the control signals that perform it on the datapath of Figure 2, and each decision box tests the LEFT and RIGHT status signals. S0 asserts Load_Display, Load_LMASK and Load_RMASK. S1 asserts shift_LMASK, shift_RMASK and Load_Display. S2 asserts shift_LMASK and Load_Display, S3 asserts shift_RMASK and Load_Display, and S4 only asserts Load_Display (the mux already selects 00000000). Unlisted signals are 0.

3.5 Control Logic (One-FF-Per-State)

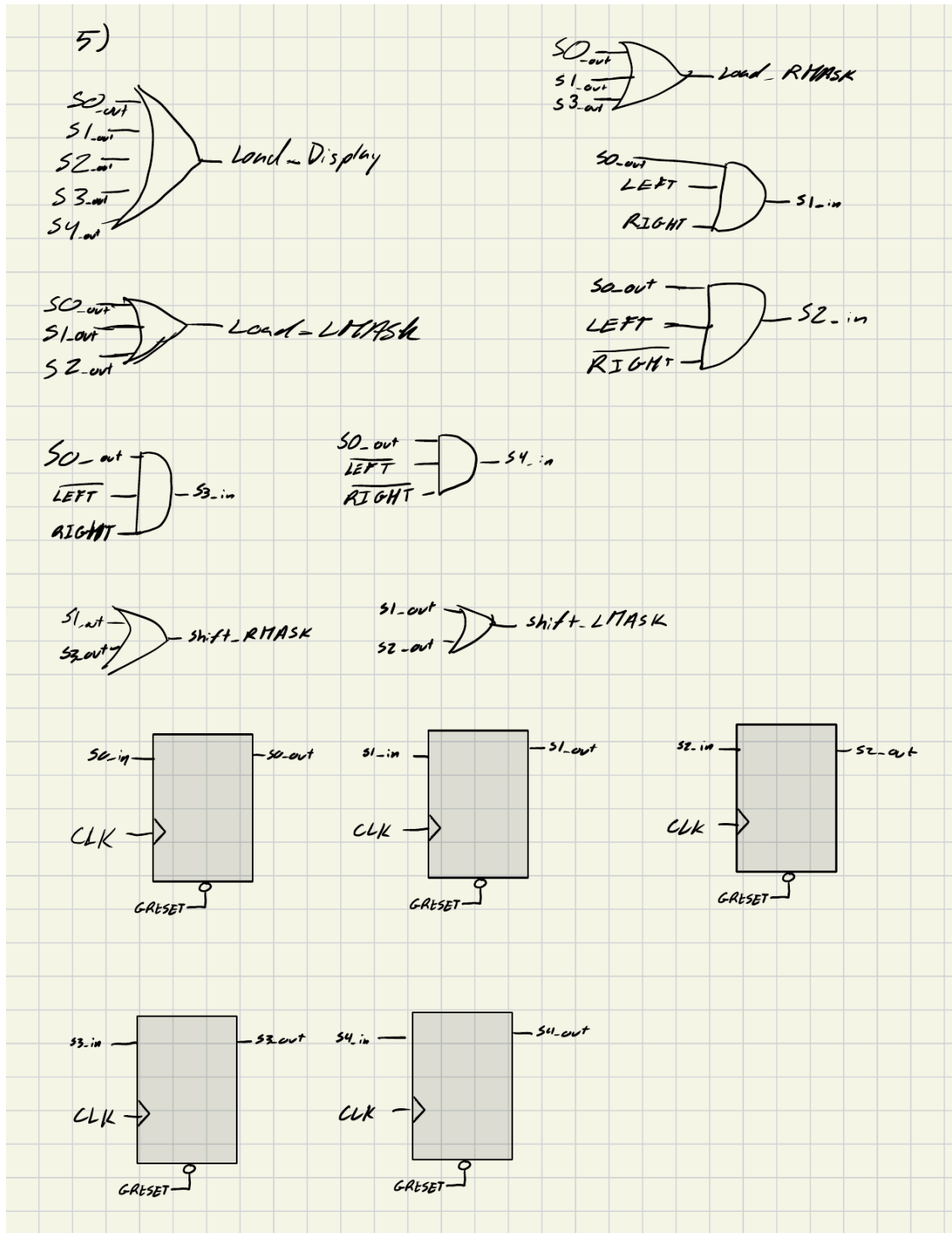


Figure 4: Control logic (one-FF-per-state)

One D flip-flop is used per state (S0 to S4), all clocked by CLK, so exactly one state is active at a time. Each control signal is the OR of the states that assert it in Figure 3:

- Load_Display = S0 + S1 + S2 + S3 + S4
- Load_LMASK = S0, shift_LMASK = S1 + S2

- $\text{Load_RMASK} = S0, \quad \text{shift_RMASK} = S1 + S3$

Every state returns to the same decision boxes, and exactly one state is always active, so the next-state inputs only depend on the switches:

- $S1_in = \text{LEFT} \cdot \text{RIGHT}$
- $S2_in = \text{LEFT} \cdot \text{RIGHT}'$
- $S3_in = \text{LEFT}' \cdot \text{RIGHT}$
- $S4_in = \text{LEFT}' \cdot \text{RIGHT}'$

Figure 4 was drawn before the VHDL was written, and it differs from the equations above in two places: it ANDs $S0_out$ into each next-state input, and it ORs $S0$, $S1$ and $S2$ into Load_LMASK (and $S0$, $S1$ and $S3$ into Load_RMASK). The equations above are the ones implemented in the VHDL.

In the VHDL (`lab1_control`), the state flip-flops are `dFF_2`, which has no reset input, so `GReset` is handled synchronously: $S0_in = \text{GReset}$, and each of $S1_in$ to $S4_in$ is also ANDed with GReset' , so the controller is in $S0$ while `GReset` is high and leaves it on the first clock edge after `GReset` is released. The mask registers use the `shiftRegister` component, where load has priority over shift. For this reason the VHDL drives the load inputs with $S0$ only and the shift inputs with $S0 + S1 + S2$ and $S0 + S1 + S3$; in $S0$ the load overrides the shift, which gives the same behaviour as the chart.

4 Implementation

4.1 Component Descriptions

The following components were written in VHDL:

- **dFF_2:** D flip-flop (course-provided, no reset) used as the one-hot state register in the control path.
- **dFlipFlop:** positive-edge-triggered D flip-flop built from six NAND-type gates, with an asynchronous active-low reset ($i_reset_bar = 0$ forces Q to 0). It is the storage element of the shift register.
- **mux4_1bit:** 1-bit 4-to-1 multiplexer written as a sum of products; select 00 picks a, 01 picks b, 10 picks c and 11 picks d.
- **shiftRegister:** generic n-bit register with one `mux4_1bit` and one `dFlipFlop` per bit. The select lines are $sel(1) = \text{shiftL} + \text{load}$ and $sel(0) = \text{shiftR} + \text{load}$, so 00 holds, 01 shifts right (bit i takes bit $i+1$), 10 shifts left (bit i takes bit $i-1$) and 11 loads d. Because load drives both select lines, it has priority over either shift. The end bits take `shiftExtension` as the incoming bit. It is intended for `LMASK` and `RMASK`.
- **lab1_control:** one-hot controller with states $S0$ to $S4$ producing the load and shift control signals.

The datapath (`DISPLAY` register, 4-to-1 display mux and OR gate) and the top-level entity connecting it to `lab1_control` were not completed. The board clock is 100 MHz, so a clock divider down to 1 Hz would also be needed before the design can run on the board. The full code of the completed files is in the Appendix.

4.2 Planned Pin Assignment

Since the top-level entity was not completed, the following pin assignment is the planned one and was not tested on the board.

Logical port	Board resource	Pin
GClock	CLK100MHZ	E3
Left	SW1	L16
Right	SW2	M13
GReset	SW3	R15
DisplayOut[7..0]	LED7..LED0	U16, U17, V17, R18, N14, J13, K15, H17

Table 2: Planned Nexys A7-100T pin assignment

5 Simulation Results

5.1 Shift Register

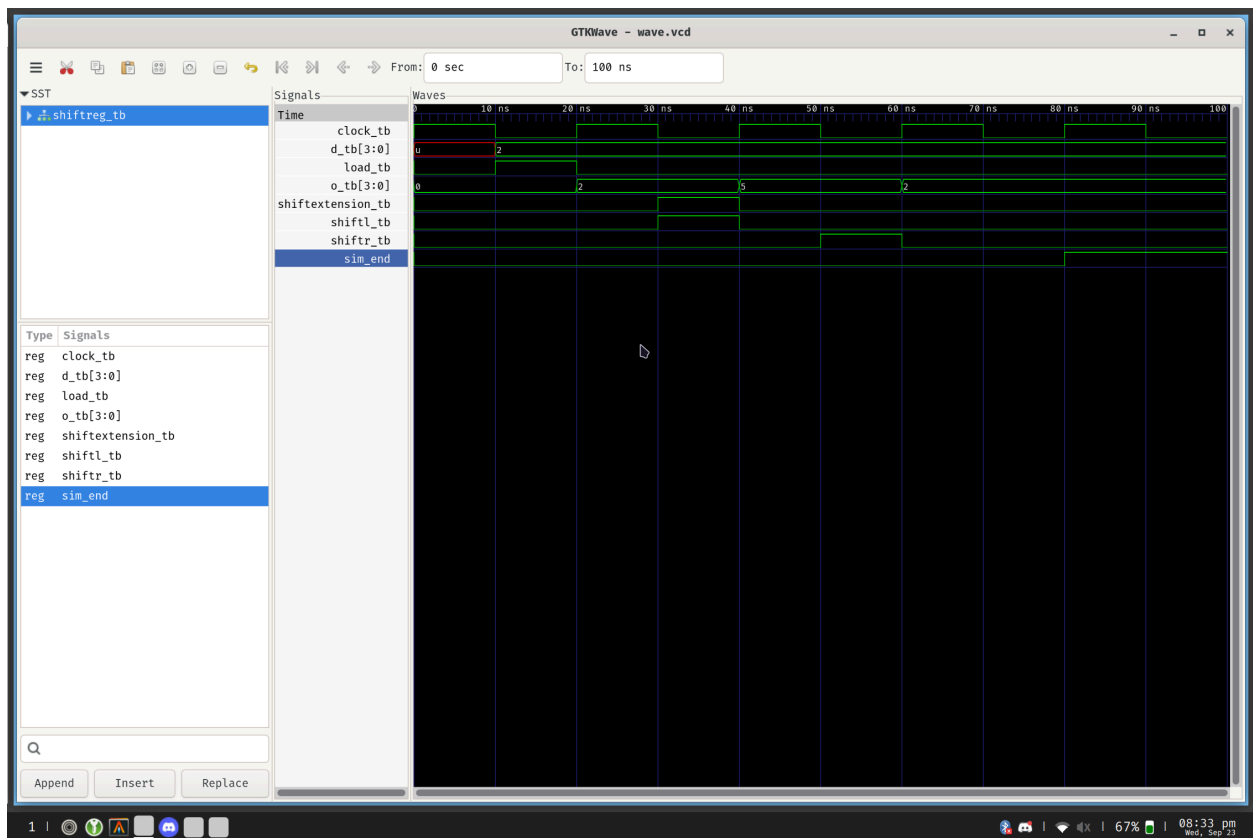


Figure 5: Shift register simulation (4 bits, clock period 20 ns)

The testbench checks the three operations of the shift register. At 10 ns, `d_tb` is set to 0010 and `load_tb` goes high, so at the rising edge at 20 ns the register loads 0010. From 30 to 40 ns, `shiftl_tb` and `shiftextension_tb` are high, so at the 40 ns edge the register shifts left and the extension bit enters bit 0: 0010 becomes 0101. From 50 to 60 ns, `shiftr_tb` is high with the extension at 0, so at the 60 ns edge the register shifts right: 0101 becomes 0010. Between these operations no control signal is high, so the output holds its value. The results match the expected values.

5.2 Control Logic

No testbench was written for `lab1_control`, so no waveform is included. Tracing the VHDL equations: while `GReset` is high, only `S0` is set on the next clock edge, which asserts `Load_LMASK`, `Load_RMASK` and `Load_Display` (`shift_LMASK` and `shift_RMASK` are also high in `S0`, but load has priority in the shift register). After `GReset` is released, each switch combination sets exactly one of `S1` (`LEFT` and `RIGHT`), `S2` (`LEFT` only), `S3` (`RIGHT` only) or `S4` (neither) on the next edge, which matches the detailed ASM chart.

5.3 Full Display Controller

The datapath and top-level entity were not completed in time, so the full display controller was not simulated. Traced by hand from the ASM chart, the display should show 00000001, 00000010, 00000100 and so on with `LEFT` on, 10000000, 01000000 and so on with `RIGHT` on, and `LMASK` OR `RMASK` with both on, so the two lights move toward each other. With both switches off, the display is blank but `LMASK` and `RMASK` keep their values, so the motion resumes from where it stopped when a switch is turned back on.

6 Hardware Test

Since the top-level entity was not completed and we were not able to access the lab before the submission of the report, the design was not demonstrated on the board.

7 Design Obstacles

- **No reset on the state flip-flops:** the course-provided `dFF_2` has no reset input, so `GReset` was made synchronous by feeding it into `S0_in` and blocking `S1_in` to `S4_in` while it is high.
- **Load and shift in the same state:** the chart loads the masks in `S0` while `S1` to `S3` shift them. Giving load priority over shift in `shiftRegister` let the shift signals stay simple without conflicting with the load in `S0`.
- **Time and lab access:** the datapath, top-level entity, clock divider and board test were not finished before the deadline.

8 Conclusion

The light display controller was designed with the five steps of the ASM method: pseudocode, ASM chart, datapath, detailed ASM chart and one-FF-per-state control logic. The control path and the shift register were implemented in structural VHDL, and the shift register simulation matched the expected load, shift left and shift right results. The remaining work is the datapath, the top-level entity with a 1 Hz clock divider, a full simulation and the board demonstration.

A Appendix: VHDL Source Code

A.1 dFF_2.vhd

```
-----
-- Title       : Type D Flip-Flop - 2nd realization
-- Project     : VHDL Synthesis Overview
-----
-- File        : dFF_2.vhd
-- Author      : Rami Abielmona <rabiemo@site.uottawa.ca>
-- Created     : 2003/05/17
-- Last modified : 2007/09/25
-----
-- Description : This file creates a flip-flop of type D as defined in the VHDL
--               Synthesis lecture. The architecture is done at the RTL
--               abstraction level and the implementation is done in structural
--               VHDL.
-----
-- Modification history :
-- 2003.05.17  R. Abielmona    Creation
-- 2004.09.22  R. Abielmona    Ported for CEG 3550
-- 2007.09.25  R. Abielmona    Modified copyright notice
-----
-- This file is copyright material of Rami Abielmona, Ph.D., P.Eng., Chief Research
-- Scientist at Larus Technologies. Permission to make digital or hard copies of part
-- or all of this work for personal or classroom use is granted without fee
-- provided that copies are not made or distributed for profit or commercial
-- advantage and that copies bear this notice and the full citation of this work.
-- Prior permission is required to copy, republish, redistribute or post this work.
-- This notice is adapted from the ACM copyright notice.
-----

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dFF_2 IS
  PORT(
    i_d   : IN  STD_LOGIC;
    i_clock : IN  STD_LOGIC;
    o_q, o_qBar : OUT STD_LOGIC);
END dFF_2;

ARCHITECTURE rtl OF dFF_2 IS
  SIGNAL int_q : STD_LOGIC;

BEGIN

  oneBitRegister:
  PROCESS(i_clock)
  BEGIN
    IF (i_clock'EVENT and i_clock = '1') THEN
      int_q <= i_d;
    END IF;
  END PROCESS oneBitRegister;

  -- Output Driver
```

```

o_q   <= int_q;
o_qBar <= not(int_q);

END rtl;

```

A.2 lab-1-control.vhdl

```

library ieee;
use ieee.std_logic_1164.all;

entity lab1_control is
  port (
    i_clock: in std_logic;
    i_reset: in std_logic;
    i_left : in std_logic;
    i_right : in std_logic;

    o_loadDisplay : out std_logic;
    o_loadLMask: out std_logic;
    o_loadRMask : out std_logic;
    o_shiftLMask: out std_logic;
    o_shiftRMask : out std_logic
  );
end lab1_control;

architecture structural of lab1_control is
  signal s0in, slin, s2in, s3in, s4in : std_logic;
  signal s0out, slout, s2out, s3out, s4out : std_logic;

  component dff_2 is
    port(
      i_d : in std_logic;
      i_clock : in std_logic;
      o_q, o_qbar : out std_logic);
  end component;

begin
  s0in <= i_reset;
  slin <= i_left and i_right and not i_reset;
  s2in <= i_left and not i_right and not i_reset;
  s3in <= not i_left and i_right and not i_reset;
  s4in <= not i_left and not i_right and not i_reset;

  o_loadLMask <= s0out;
  o_loadRMask <= s0out;

  o_loadDisplay <= s0out or slout or s2out or s3out or s4out;
  o_shiftLMask <= s0out or slout or s2out;
  o_shiftRMask <= s0out or slout or s3out;

  s0 : dff_2
  port map (
    i_d => s0in,
    i_clock => i_clock,

```

```

        o_q => s0out
    );

s1 : dff_2
port map (
    i_d => s1in,
    i_clock => i_clock,
    o_q => s1out
);

s2 : dff_2
port map (
    i_d => s2in,
    i_clock => i_clock,
    o_q => s2out
);

s3 : dff_2
port map (
    i_d => s3in,
    i_clock => i_clock,
    o_q => s3out
);

s4 : dff_2
port map (
    i_d => s4in,
    i_clock => i_clock,
    o_q => s4out
);

end architecture;

```

A.3 dFlipFlop.vhdl

```

library ieee;
use ieee.std_logic_1164.all;
entity dFlipFlop is
    port (
        i_d      : in  std_logic;
        i_clk     : in  std_logic;
        i_reset_bar : in std_logic;
        o_q, o_qBar : out std_logic
    );
end dFlipFlop;

architecture structural of dFlipFlop is
    signal a : std_logic := '0';
    signal b, c : std_logic := '1';
    signal d : std_logic := '1';
    signal int_q : std_logic := '0';
    signal int_qBar : std_logic := '1';
begin
    a <= d nand b;

```

```

b <= not (a and i_clk and i_reset_bar);
c <= not (b and d and i_clk);
d <= not (c and i_d and i_reset_bar) after 0.5 ns;

int_q <= b nand int_qBar;
int_qBar <= not (int_q and c and i_reset_bar);

o_q <= int_q;
o_qBar <= int_qBar;
end architecture;

```

A.4 mux4_1bit.vhdl

```

-- ===== 4x1 MUX
library ieee;
use ieee.std_logic_1164.all;
entity mux4_1bit is
    port (
        a, b, c, d : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        o : out std_logic
    );
end mux4_1bit;

architecture structural of mux4_1bit is
    -- s1, s0
    -- a for 00
    -- b for 01
    -- c for 10
    -- d for 11
begin
    o <= (a and (not sel(1)) and (not sel(0))) or
        (b and (not sel(1)) and sel(0)) or
        (c and sel(1) and (not sel(0))) or
        (d and sel(1) and sel(0));
end structural;

```

A.5 shiftReg-nbit.vhdl

```

-- ===== N-BIT SHIFT REGISTER
library ieee;
use ieee.std_logic_1164.all;

entity shiftRegister is
    generic ( n : integer := 4 );
    port (
        clock, reset : in std_logic;
        load, shiftL, shiftR : in std_logic;
        shiftExtension : in std_logic;

        d : in std_logic_vector(n-1 downto 0);
        o : out std_logic_vector(n-1 downto 0)
    );

```

```

end shiftRegister;

architecture structural of shiftRegister is
    signal int_d : std_logic_vector(n-1 downto 0);
    signal int_q : std_logic_vector(n-1 downto 0);
    signal int_sel : std_logic_vector(1 downto 0);

    component dFlipFlop is
        port (
            i_d      : in  std_logic;
            i_clk     : in  std_logic;
            i_reset_bar : in std_logic;
            o_q, o_qBar : out std_logic
        );
    end component;

    component mux4_1bit is
        port (
            a, b, c, d : in std_logic;
            sel : in std_logic_vector(1 downto 0);
            o : out std_logic
        );
    end component;
begin

    o <= int_q;

    -- == MUXES have to separate out LSB and MSB to add optional shift extension
    -- LOAD takes priority over shift
    int_sel(1) <= shiftL or load;
    int_sel(0) <= shiftR or load;

    muxes: for i in n-2 downto 1 generate
    begin
        mux_i : mux4_1bit
        port map (
            a => int_q(i), -- hold
            b => int_q(i+1), -- shift L
            c => int_q(i-1), -- shift R
            d => d(i), -- load
            sel => int_sel,
            o => int_d(i)
        );
    end generate;

    -- rightmost
    mux_lsb: mux4_1bit
    port map (
        a => int_q(0),
        b => int_q(1),
        c => shiftExtension,
        d => d(0),
        sel => int_sel,
        o => int_d(0)
    );

```

```

-- leftmost
mux_msb: mux4_lbit
port map (
  a => int_q(n-1),
  b => shiftExtension,
  c => int_q(n-2),
  d => d(n-1),
  sel => int_sel,
  o => int_d(n-1)
);

bits: for i in n-1 downto 0 generate
begin
  b_i : dFlipFlop
  port map (
    i_clk => clock,
    i_reset_bar => reset,
    i_d => int_d(i),
    o_q => int_q(i)
  );
end generate;

end structural;

-- ==== SHIFT REGISTER TEST BENCH
library ieee;
use ieee.std_logic_1164.all;
entity shiftReg_TB is
  generic ( n : integer := 4 );
end shiftReg_TB;

architecture testbench of shiftReg_TB is
  signal clock_tb : std_logic := '0';
  signal d_tb, o_tb: std_logic_vector(n-1 downto 0);
  signal load_TB, shiftL_TB, shiftR_TB, shiftExtension_TB: std_logic := '0';
  signal sim_end : BOOLEAN := false;
  constant CLOCK_PERIOD : time := 20 ns;

  component shiftRegister is
    generic ( n : integer := 4 );
    port (
      clock, reset : in std_logic;
      load, shiftL, shiftR : in std_logic;
      shiftExtension : in std_logic;

      d : in std_logic_vector(n-1 downto 0);
      o : out std_logic_vector(n-1 downto 0)
    );
  end component;
begin
  dut: shiftRegister
  port map (
    clock => clock_tb,
    load => load_TB,
    shiftL => shiftL_tb,

```

```

    shiftR => shiftR_tb,
    shiftExtension => shiftExtension_tb,

    d => d_tb,
    o => o_tb,

    reset => '1'
);

clock_process:
process begin
    while (not sim_end) loop
        clock_tb <= '1';
        wait for clock_period / 2;
        clock_tb <= '0';
        wait for clock_period / 2;
    end loop;
    wait;
end process clock_process;

stimulus:
process begin
    wait for clock_period / 2;
    d_tb <= "0010"; load_TB <= '1';
    wait for clock_period / 2; -- clock rising edge
    load_TB <= '0';
    wait for clock_period / 2; -- falling
    assert o_tb = "0010" report "load failed";

    shiftL_TB <= '1'; shiftExtension_TB <= '1';
    wait for clock_period / 2; -- rising
    shiftL_TB <= '0'; shiftExtension_TB <= '0';
    wait for clock_period / 2; -- rising
    assert o_tb = "0101" report "shift failed";

    wait for clock_period / 2;
    sim_end <= true;
    wait;
end process stimulus;
end testbench;

```